

CSC Problem Set 01 +X Report

Name: Nour Haissam Mahmoud

Email: nhmahmou@ncsu.edu

Section: 001

Collaborators: N/A

Summary:

Since this problem set is focused on designing and implementing a visual simulation of a robot cleaner navigating a grid environment, I decided to dive deeper into the GUI to make it more functional. I focused on enhancing the GUI by adding various interactive buttons to control the simulation in real time. Pause to temporarily stop the simulation; Reset to restart the simulation with the current map; Restart to reload the simulation from scratch; Load Map to allow users to load custom environment maps; and Speed Up and Slow Down to adjust the speed of the simulation. Additionally, a Load Config button was added to let users load configuration files to change settings like TILESIZE, DELAY, and ITERATIONS. The GUI was also designed to dynamically adjust the frame and grid size to reflect the loaded configurations and maps. Finally, I added real time statistics updates to show the percentage of tiles that have been cleaned. All of these enhancements provided greater control over the simulation to make it easier to test different scenarios.

"+X" Concept:

The implementation of the GUI enhancements in the simulation was driven by the need to improve user interaction and control over the simulation. By adding functional buttons such as Start, Pause, Reset, Restart, Load Map, and Load Config, users are now able to interact dynamically with the simulation. Each button serves a specific purpose to ensure flexibility when testing different solutions and maps. For instance, the Pause and Start buttons allow users to temporarily stop and resume the simulation without having to restart it. The Load Map button allows the user to dynamically switch between maps, enabling the user to test different scenarios without having to recompile and run the simulation. The Load Config button provides flexibility to adjust the TILESIZE, ITERATIONS, and DELAY without having to edit the codebase.

A significant design consideration was ensuring the layout adjusted properly based on user-defined configurations while maintaining a consistent and clean interface. Moreover, a real time statistics label was implemented to provide users with important performance metrics like the number of cleaned tiles and the percentage of the grid cleaned, which is the performance measure. This helps the user evaluate the effectiveness of the robot and realize in real time, what they should be adjusting to get a better performance. The implementation of the GUI relied on several essential Java APIs to create an interactive, user-friendly interface. The `java.awt.Color` API [1] was used to define a consistent color scheme for the simulation buttons, improving the overall visual appeal of the application. The `javax.swing.Timer` API [2], which was already being utilized, enabled the Start, Reset, and Restart buttons. The

javax.swing.JFileChooser API [3] allowed users to load custom maps and configuration files dynamically, providing flexibility in testing different scenarios. Moreover, the javax.swing.JLabel API [4] was utilized to display real-time statistics, such as the number of cleaned tiles and the percentage cleaned, giving users valuable insights into the robot's performance. The Oracle Swing tutorials on timers [5] and other community resources like YouTube tutorials and guides [6] were instrumental in guiding the integration of these APIs into the GUI. The result is a polished, professional simulation that is easy to interact with, visually appealing, and robust.

Technical Implementation of +X:

To enhance the simulation's functionality and interactivity, a series of buttons were added to the GUI, each designed to control specific aspects of the simulation. The **Start** button initiates the simulation by starting the timer. The **Pause** button allows users to temporarily halt the simulation by stopping the timer. The **Reset** button resets the simulation to the current environment, ensuring the robot returns to its starting position, while keeping the map unchanged. The **Restart** button provides similar functionality but automatically resumes the simulation without requiring the user to press **Start** again.

```
//STARTS THE SIMULATION - TIMER
public void startSimulation() {
    if (!timer.isRunning()) {
        statsLabel.setText("Simulation Running...");
        timer.start();
    }
}

//PAUSES THE SIMULATION - TIMER
public void pauseSimulation() {
    if (timer.isRunning()) {
        timer.stop();
        statsLabel.setText("Simulation Paused");
    }
}

//RESETS THE SIMULATION BY RESETTNG THE TIMER, ROBOT POSITION, AND ENVIRONMENT
public void resetSimulation() {
    timer.stop();
    this.timeStepCount = 0;
    this.env = new Environment(currentMap);
    this.robots = env.getRobots();
    statsLabel.setText("Simulation Reset");
    initializeTimer();
    repaint();
}

//RESETS THE SIMULATION BY RESETTNG THE TIMER, ROBOT POSITION, ENVIRONMENT, AND STARTING THE TIMER
public void restartSimulation() {
    timer.stop();
    this.timeStepCount = 0;
    this.env = new Environment(currentMap); // Restart with the current map
    this.robots = env.getRobots();
    statsLabel.setText("Simulation Restarted");
    initializeTimer();
    repaint();
    timer.start();
}
```

Additional controls include **Speed Up** and **Slow Down**, which adjust the timer delay by either adding to it to slow down or removing from it to speed up, allowing users to observe the simulation at different speeds. The **Load Map** button leverages boilerplate code using JFileChooser to allow users to load a new map, ensuring the grid dimensions and tile size adjust to match the new environment. Similarly, the **Load Config** button lets users load configuration files to modify settings like the tile size, iterations, and delay. Each of these buttons updates the GUI and the underlying simulation environment to reflect the changes. For instance, the **stats label** is updated in real time with messages like "Simulation Running," "Simulation Paused," or "Simulation Reset" to keep users informed of current events.

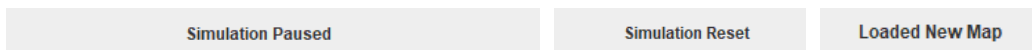
Evaluation and Results:

The addition of "+X" significantly improved the usability and functionality of the GUI. Below is a breakdown of the enhancements and their impact, supported by visuals:

1. Comparison of GUI Interfaces

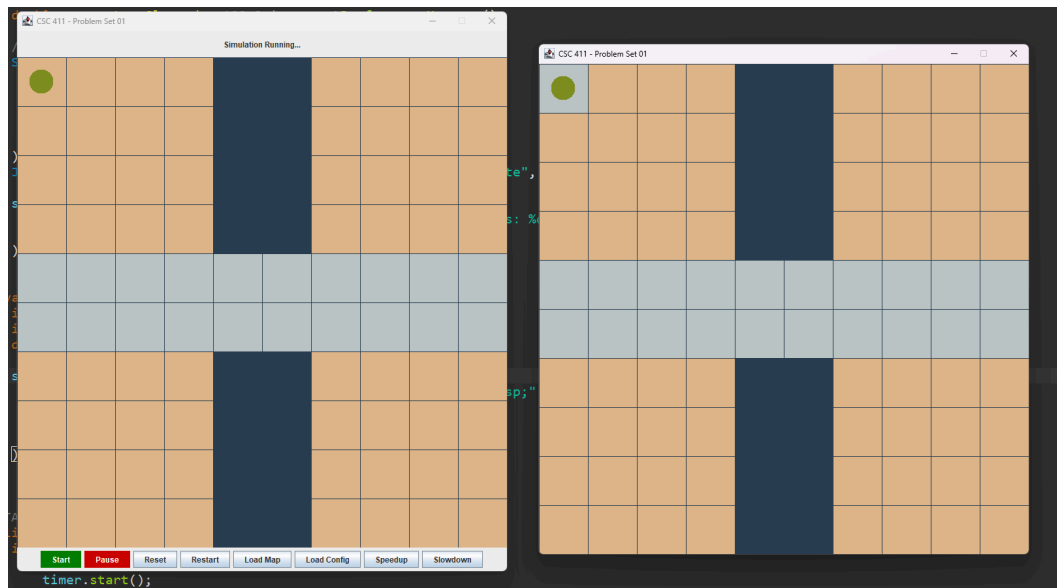
A direct comparison of the modified GUI (left) and the professor's original design (right) highlights the differences:

- **Enhanced Usability:** The new top-notch section displays key statistics, providing users with at-a-glance information like simulation status and updates such as "Paused," "Reset," or "Loaded New Map."



- **Improved Layout:** The new interface adds and organizes labels and controls intuitively, reducing user confusion, and adding usability.

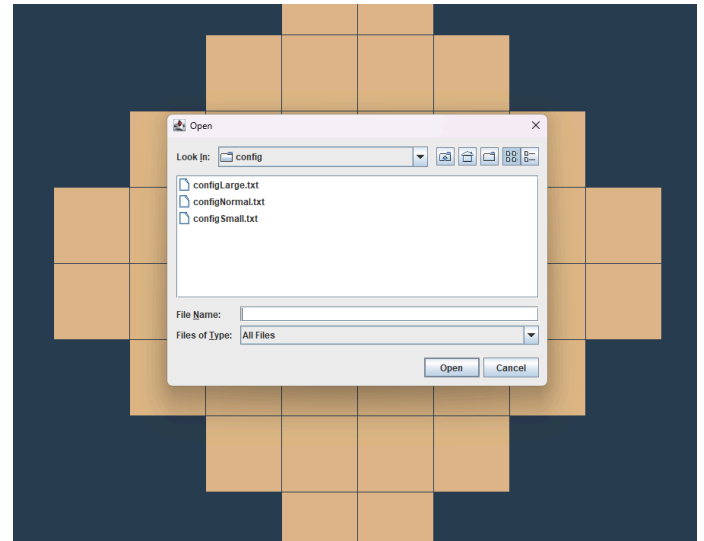
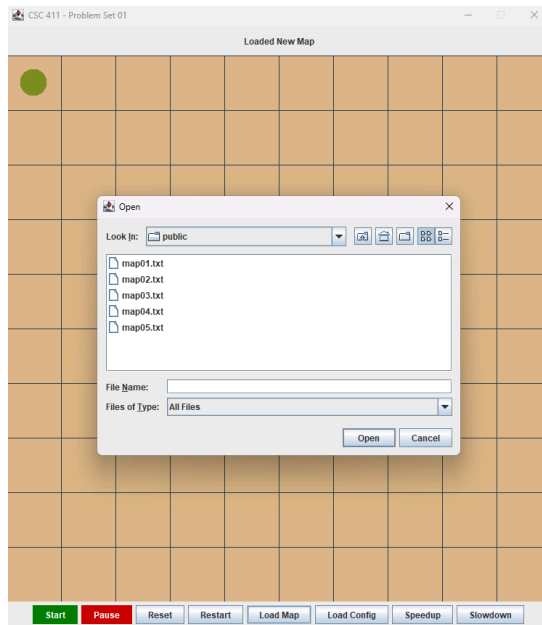
Feature	Previous GUI	Modified GUI
Simulation Status & Statistics	Absent	Clearly Visible
Map Loading & Configuration Loading	Absent	Detailed and dynamic
Control Options	Static through code edits	Dynamically adjustable



2. File Chooser Enhancements

The integration of file choosers into the GUI eased workflows:

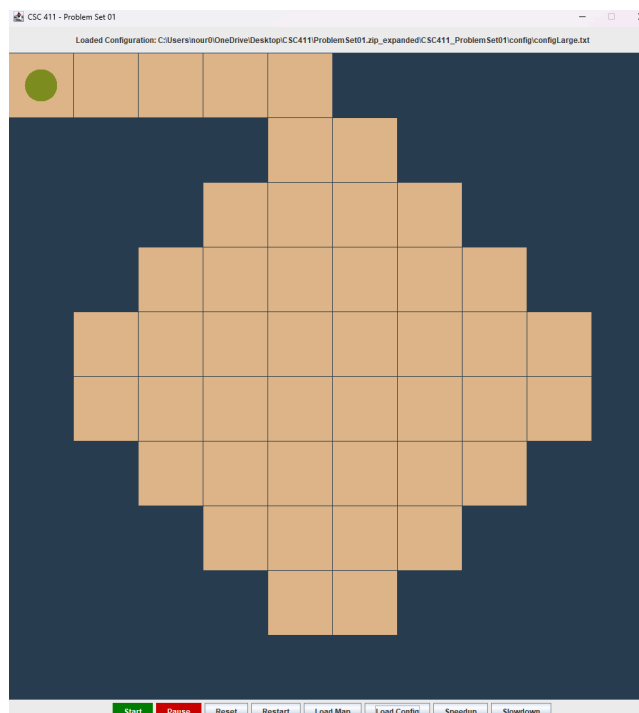
- **Map File Loader:** Users can now seamlessly select map files, as demonstrated in the screenshot of the file chooser interface.
- **Configuration File Loader:** Users can select configuration files to dynamically adjust the iterations, tile size, delay, and debug information.



3. New Large Configuration

The screenshot of the large configuration highlights the flexibility of the updated design:

- The interface accommodates larger datasets without clutter.
- Labels and statistics remain visible and accessible, ensuring that the simulation remains user-friendly.



Overall, the implemented improvements significantly enhanced the usability and performance of the GUI. By adding user-friendly features such as a file chooser and clear labels for simulation status and statistics, the program is now more intuitive and easier to navigate. Ultimately, the updates reduced the need to change the code when testing and made the testing process faster and more efficient.

Works Referenced:

1. Oracle, *Class Color: Java Platform SE 8*. 2015. Accessed: Jan. 18, 2025. [Online]. Available: <https://docs.oracle.com/javase/8/docs/api/java/awt/Color.html>
2. Oracle, *Class Timer: Java Platform SE 8*. 2015. Accessed: Jan. 18, 2025. [Online]. Available: <https://docs.oracle.com/javase/8/docs/api/java/util/Timer.html>
3. Oracle, *Class JFileChooser: Java Platform SE 8*. 2015. Accessed: Jan. 18, 2025. [Online]. Available: <https://docs.oracle.com/javase/8/docs/api/javax/swing/JFileChooser.html>
4. Oracle, *Class JLabel: Java Platform SE 8*. 2015. Accessed: Jan. 18, 2025. [Online]. Available: <https://docs.oracle.com/javase/8/docs/api/javax/swing/JLabel.html>
5. Oracle, *How to Use Swing Timers*. 2015. Accessed: Jan. 18, 2025. [Online]. Available: <https://docs.oracle.com/javase/tutorial/uiswing/misc/timer.html>
6. Duquesne University, *How to Use Swing Timers – Archive*. 2015. Accessed: Jan. 18, 2025. [Online]. Available: <https://www.mathcs.duq.edu/simon/Java6/uiswing/misc/timer.html>
7. Drifter1, *Programming Java GUI Examples*. 2018. Accessed: Jan. 18, 2025. [Online]. Available: <https://steemit.com/programming/@drifter1/programming-java-gui-examples>